

Module - Plateformes de Développement

Spring

Support de cours

Par :

Dr. Ing. Faouzi JAIDI

faouzi.jaidi@gmail.com

2021-2022

Plan

- 1. Introduction
- 2. Maven
- 3. Logs : Log4J
- 4. Tests: JUnit
- 5. Tests & Qualité : Sonar
- 6. IOC & ID
- 7. Spring Boot
- 8. Spring Data JPA
- 9. Spring MVC REST
- 10. JSF
- 11. AOP
- 12. Projet
- 13. Spring Security
- 14. Extensions

Spring

Journalisation: Log4J



Journalisation

□ Logging / Journalisation

○ *Présentation*

- Consiste à ajouter des traitements dans les applications pour permettre l'émission et le stockage de messages suite à des événements.
- Utile pour tous types d'applications:
 - Conserver une trace des exceptions,
 - Conserver une trace des différents événements anormaux ou normaux liés à l'exécution de l'application.
- Permet de gérer les messages émis par une application et leur exploitation immédiate ou a posteriori.

Journalisation

o Utilités

- Débogage
 - Faciliter le débogage et simplifier la recherche des sources des anomalies.
 - Traces
 - Collecter des traces d'exécution et d'utilisation (démarrage/arrêt, informations, avertissements, erreurs d'exécution, ...).
 - Vérification
 - Vérifier le flux des traitements exécutés : traces des entrées/sorties dans les méthodes, affichage de la pile d'appels.
- *L'importance du logging croît avec la taille et la complexité de l'application.*

Journalisation

○ Frameworks

— Plusieurs frameworks de journalisation sont disponibles pour java:

- Log4j
- Java Logging
- Jlog
- Protomatter
- SLF4J
- LogBack

➤ *Log4j (projet open source distribué sous la licence Apache) est sûrement l'API la plus répandue et la plus populaire.*

Log4J

□ Présentation

- Un API de logging « open source »;
<http://logging.apache.org/log4j/>
- Un projet distribué sous la licence *Apache Software* et maintenu par le groupe Jakarta;
 - *Simplicité de mise en œuvre*
 - *Facilité d'utilisation*
 - *Nombreuses fonctionnalités*
 - *Fiabilité*
 - *Évolutivité*

Standard pour le logging.

□ Caractéristiques et Composants

○ *Caractéristiques*

- Utilisation d'une hiérarchie de loggers basée sur leurs noms,
- Support en standard de plusieurs niveaux de gravité,
- Configuration externalisable dans un fichier (format .properties ou XML),
- Thread-safe,
- Optimisé pour réduire les temps de traitements,
- Prise en charge des exceptions associables aux messages,
- Support de nombreuses cibles de destination des messages,
- Extensible,
- Etc.

□ Caractéristiques et Composants

○ *Caractéristiques*

- Utilisation d'une hiérarchie de loggers basée sur leurs noms,
- Support en standard de plusieurs niveaux de gravité,
- Configuration externalisable dans un fichier (format .properties ou XML),
- Thread-safe,
- Optimisé pour réduire les temps de traitements,
- Prise en charge des exceptions associables aux messages,
- Support de nombreuses cibles de destination des messages,
- Extensible,
- Etc.

Log4J

o Composants

- Log4j utilise trois composants principaux:

Loggers

- Permettent de gérer les messages.
- Invoqués pour émettre un message avec un niveau de gravité associé.

Appenders

- Représentent les flux qui vont recevoir les messages de log.
- Utilisés pour envoyer le message à une cible de stockage (console, fichier, base de données, Syslog, email, ...).

Layouts

- Permettent de formater le contenu des messages de log.

□ Niveaux de Log / gravité

– Niveaux de gravité standard possédant un **ordre hiérarchique** :

- **TRACE** : messages de traces d'exécution (depuis la version 1.2.12)
- **DEBUG** : messages de débogage.
- **INFO** : messages d'information.
- **WARN** : messages d'avertissement.
- **ERROR** : messages d'erreur.
- **FATAL** : messages liés à un arrêt imprévu de l'application.

➤ **OFF** : aucun niveau de gravité n'est pris en compte.

➤ **ALL** : tous les niveaux de gravité sont pris en compte.

Log4J

<i>Niveau de gravité</i>	<i>Exemple d'utilisation</i>
TRACE	Entrée et sortie de méthodes
DEBUG	Affichage de valeur de données
INFO	Chargement d'un fichier de configuration, début et fin d'exécution d'un traitement long
WARN	Erreur de login, données invalides
ERROR	Toutes les exceptions capturées qui n'empêchent pas l'application de fonctionner
FATAL	Indisponibilité d'une base de données, toutes les exceptions qui empêchent l'application de fonctionner

Log4J

□ Appenders

- L'interface ***org.apache.log4j.Appender*** désigne un flux qui représente le log et se charge de l'envoi de messages formatés dans le flux.
- Un logger peut posséder plusieurs appenders. Si le logger décide de traiter la demande de message, le message est envoyés à chacun des appenders.
- Log4J propose plusieurs appenders. Chaque appender possède des paramètres de configuration dédiés.
 - AsyncAppender,
 - JMSAppender,
 - JDBCAppender,
 - SyslogAppender,
 - RollingFileAppender,
 - SMTPAppender,
 - ...

Log4J

o *AsyncAppender*

- La classe ***org.apache.log4j.AsyncAppender*** envoie les messages vers différents appenders de façon périodique et asynchrone.
- Un tag fils ***<appender-ref>*** permet de préciser un appender vers lequel les messages seront envoyés.
- L'attribut ***ref*** permet de préciser le nom de l'appender concerné.
- Exemple d'ajout d'un appender dans log4j.xml:

```
<root>
  <level value="INFO" />
  <appender-ref ref="console" />
  <appender-ref ref="file" />
</root>
```

□ Layouts

- La classe ***org.apache.log4j.Layout*** permet de définir le format du fichier de log.
- Un layout est associé à un appender lors de son instantiation.
- Le PatternLayout permet de préciser le format du message grâce à un motif dont certaines séquences seront dynamiquement remplacées par leurs valeurs correspondantes à l'exécution.

Log4J

<i>Motif</i>	<i>Rôle</i>
%c	Le nom de la catégorie ou du logger qui a émis le message
%C	Le nom de la classe qui a émis le message : l'utilisation de ce motif est coûteuse en ressources
%d	Le timestamp de l'émission du message. Il est possible de fournir un format pour la date/heure en utilisant les motifs de la classe SimpleDateFormat. Exemple : %d{dd MMM yyyy HH:MM:ss }
%m	Le message
%n	Un saut de ligne dépendant de la plate-forme
%p	Le niveau de gravité du message
%r	Le nombre de millisecondes écoulées entre le lancement de l'application et l'émission du message
%t	Le nom du thread
%x	NDC (Nested Diagnostic Context) du thread. Ceci est particulièrement utile pour les applications de type web.
%%	Le caractère %
%L	Le numéro de ligne dans le code émettant le message : l'utilisation de ce motif est coûteuse en ressources
%F	Le nom du fichier émettant le message : l'utilisation de ce motif est coûteuse en ressources
%M	Le nom de la méthode émettant le message : l'utilisation de ce motif est coûteuse en ressources
%I	Des informations sur l'origine du message dans le code source (C'est un raccourci dépendant de la JVM qui correspond généralement à %C.%M(%F:%L)): l'utilisation de ce motif est coûteuse en ressources

<i>Motif</i>	<i>Rôle</i>
<code>%#</code>	Aucun formatage (par défaut)
<code>%n#</code>	Alignement à droite, des blancs sont ajoutés si la taille du motif est inférieure à n caractères
<code>%-n#</code>	Alignement à gauche, des blancs sont ajoutés si la taille du motif est inférieure à n caractères
<code>%.n</code>	Tronque le motif s'il est supérieur à n caractères
<code>%-n.n#</code>	Alignement à gauche, taille du motif obligatoirement de n caractères (troncature ou complément avec des blancs)

Log4J

□ Utilisation

- ① Ajouter la dépendance / le fichier **log4j-1.2.x.jar** à l'application:
 - ✓ *[Maven: pom.xml].*
- ② Créer un fichier de configuration [**log4j.xml** au niveau **src/main/resources**]:
 - ✓ *Configuration des loggers, définition des appenders, association des appenders aux loggers avec un layout.*
- ③ Au niveau du classes:
 - ✓ *Récupérer une instance du logger relative à la classe,*
 - ✓ *Utiliser l'API pour émettre un message associé à un niveau de gravité.*

Log4J

0 *Dépendance*

1

```
<!-- https://mvnrepository.com/artifact/log4j/log4j
-->
<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.17</version>
</dependency>
```

Log4J

0 log4j.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE log4j:configuration SYSTEM "http://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/xml/doc-files/log4j.dtd">
<log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/">
  <appender name="console" class="org.apache.log4j.ConsoleAppender">
    <layout class="org.apache.log4j.PatternLayout"> <param name="ConversionPattern" value="%d{yyyy/MM/dd HH:mm:ss} [%p] %l:%L - %m%n" />
    </layout>
  </appender>
  <appender name="fichierLog"
    class="org.apache.log4j.RollingFileAppender">
    <param name="maxFileSize" value="1024KB" />
    <param name="maxBackupIndex" value="5" />
    <param name="File" value="G:/COURS/Spring 2021-2022/Journalisation/p2-demo-log4j.log" />
    <param name="threshold" value="info" />
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern"
        value="%d{yyyy-MM-dd HH:mm:ss.SSS} %-8p [%t]:%C - %m%n" />
    </layout>
  </appender>
  <appender name="fichierDebug"
    class="org.apache.log4j.RollingFileAppender">
    <param name="maxFileSize" value="10MB" />
    <param name="maxBackupIndex" value="2" />
    <param name="File" value="G:/COURS/Spring 2021-2022/Journalisation/p2-demo-log4j_debug.log" />
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern"
        value="%d{yyyy-MM-dd HH:mm:ss.SSS} %-8p [%t]:%C - %m%n" />
    </layout>
    <filter class="org.apache.log4j.varia.LevelMatchFilter">
      <param name="levelToMatch" value="DEBUG" />
    </filter>
    <filter class="org.apache.log4j.varia.DenyAllFilter"/>
  </appender>
  <root>
    <priority value="debug"></priority>
    <appender-ref ref="fichierLog" />
    <appender-ref ref="fichierDebug" />
    <appender-ref ref="console" />
  </root>
</log4j:configuration>
```

2

0 Exemple d'utilisation

```
import org.apache.log4j.Logger;

public class ProgramDemo {
    private static final Logger LG = Logger.getLogger(ProgramDemo.class);
    int x = 5;

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        ProgramDemo pd = new ProgramDemo();
        pd.calculer();
    }

    public void calculer() {
        LG.info("Début de la méthode calculer (): ");
        try {
            LG.debug("Lancement d'une opération de division : ");
            for (int i = 1; i < 5; i++) {
                x = x / i;
                LG.debug("Résultat de l'opération de division : " + x);
            }
            LG.debug("Fin de l'opération de division : ");
        } catch (ArithmeticException e) {
            LG.error("Erreur au niveau de la méthode calculer() : " + e);
        }
        LG.info("Fin de la méthode calculer () sans erreur : ");
    }
}
```

3



Spring

TP

